

REVIEW / PAIZA

GAME PROGRAMMING COURSE — C# BASICS

C# の復習と Paiza 演習

入力の受け取り方を押さえて、例題をどんどん解こう

対象：1年生（クラスまで既習・2週目）

1

LESSON 1

復習プログラムの解説

目標：Paiza の問題を解くための「書き方の型」を思い出す

LESSON 1

このコマの目標

- Paiza のプログラムの「いつもの形」を書けるようになる
- Paiza の入力（標準入力）を 4 つのパターンで受け取れるようになる
- 変数・計算・条件分岐・繰り返し・配列・関数・クラスをざっと思い出す
- このあと例題を解くときに、このページたちを見返せるようにしておく

Paiza のプログラムの形

- Paiza の問題は、毎回この形から書き始める
- 上の 3 行の using は「常を書く」。処理は Main の {} の中に書いていく

C# Program.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  class Program
5  {
6      static void Main()
7      {
8          // ここに処理を書く
9      }
10 }
```

POINT

using の 3 行 → class Program → static void Main() の順番。迷ったらこの形に戻ろう。

常に書く using 3 行の意味

- using は「この工具箱を使います」という宣言。書いておくと便利な機能がすぐ使える

書くもの	使えるようになるもの	どこで使う？
using System;	Console（入力・出力）、int.Parse など	ほぼすべての問題
using System.Collections.Generic;	List などの便利な入れ物	今後の問題で使う
using System.Linq;	Select / ToArray など	スペース区切りの数値の受け取り

POINT

意味は今すぐ全部覚えなくて OK。「Paiza では毎回 3 行とも書く」と覚えておこう。

出力：Console.WriteLine

- WriteLine：表示して改行する
- Write：表示するだけ（改行しない）
- Paiza の出力は WriteLine が基本
- + で文字や数値をつなげて表示できる

C# Program.cs

```
1 Console.WriteLine("Hello");
2 Console.WriteLine(10 + 5);
3 Console.Write("A");
4 Console.Write("B");
5 Console.WriteLine();
6 Console.WriteLine("HP:" + 100);
```

実行結果

```
Hello
15
AB
HP:100
```

注意

Paiza では「余計な文字・空白」があると不正解。出力例とまったく同じ形で出すこと。

変数と型

- 変数=値を入れておく名前付きの箱。型で「何を入れる箱か」を決める

型	入れるもの	例
int	整数	10, -3
double	小数	3.14
string	文字列	"paiza"
bool	true / false	true

C# Program.cs

```
1 int hp = 100;
2 double rate = 1.5;
3 string name = "ゆうた";
4 bool isAlive = true;
5 hp = hp - 30;
6 Console.WriteLine(name + " " + hp);
```

POINT

型 変数名 = 値; の形。一度作った変数は 変数名 = 値; で中身を入れ替えられる。

計算（演算子）

記号	意味	例 → 結果
+	足し算	$7 + 2 \rightarrow 9$
-	引き算	$7 - 2 \rightarrow 5$
*	掛け算	$7 * 2 \rightarrow 14$
/	割り算	$7 / 2 \rightarrow 3$
%	割った余り	$7 \% 2 \rightarrow 1$

C# Program.cs

```
1 int a = 7;
2 int b = 2;
3 Console.WriteLine(a / b);
4 Console.WriteLine(a % b);
5 a += 3; // a = a + 3 と同じ
6 a++; // 1 増やす
7 Console.WriteLine(a);
```

実行結果

```
3
1
11
```

注意

int どうしの割り算は小数点以下が切り捨て（ $7 / 2$ は 3）。% は「偶数・奇数」の判定によく使う。

一行の受け取り

- `Console.ReadLine()` で、入力を 1 行まるごと文字列として受け取る
- 1 回呼ぶごとに 1 行ずつ読み進む

C# Program.cs

```
1 string text = Console.ReadLine();  
2 Console.WriteLine(text);
```

入力例

paiza

text の中身

"paiza"

出力

paiza

POINT

`ReadLine` で受け取ったものは、数字に見えても必ず `string`（文字列）。

一行に数値が 1 つ

- 文字列で受け取ってから `int.Parse` で整数に変換する
- 変換しないと計算できない ("`5`" + 1 は "`51`" になってしまう)

C# Program.cs

```
1 string text = Console.ReadLine();  
2 int number = int.Parse(text);  
3 Console.WriteLine(number * 2);
```

入力例

5

text の中身

"5" (文字列)

number の中身

5 (整数)

出力

10

POINT

「受け取る → `int.Parse` で変換」の 2 ステップをセットで覚えよう。

一行にスペース区切りの文字

- `Split(" ")` で、スペースのところで切り分けて `string` 型の配列にする
- 切り分けた 1 つ 1 つは `words[0]`, `words[1]`, ... で取り出せる

C# Program.cs

```
1 string text = Console.ReadLine();
2 string[] words = text.Split(" ");
3 Console.WriteLine(words[0]);
4 Console.WriteLine(words.Length);
```

入力例

apple banana cherry

words の中身

[0] "apple"
[1] "banana"
[2] "cherry"

出力

apple
3

一行にスペース区切りの数値

- Split で切り分け → Select(int.Parse) で全部を整数に → ToArray() で配列に

C# Program.cs

```
1 string text = Console.ReadLine();  
2 int[] numbers = text.Split(" ").Select(int.Parse).ToArray();  
3 Console.WriteLine(numbers[0] + numbers[1]);
```

① Split(" ")

"3" "5" "7"

② Select(int.Parse)

3 5 7 (全部 int に)

③ ToArray()

int[] { 3, 5, 7 }

出力

8

POINT

Select と ToArray は using System.Linq; が必要。だから 3 行の using は「常を書く」。

入力の受け取り方 一覧（演習中に見返そう）

入力の形	例	書き方
一行（文字）	paiza	<code>string text = Console.ReadLine();</code>
一行に数値 1 つ	5	<code>string text = Console.ReadLine(); int number = int.Parse(text);</code>
スペース区切りの文字	a b c	<code>string text = Console.ReadLine(); string[] words = text.Split(" ");</code>
スペース区切りの数値	3 5 7	<code>string text = Console.ReadLine(); int[] numbers = text.Split(" ").Select(int.Parse).ToArray();</code>

POINT

問題文の「入力される値」を見て、1 行ずつ「どの形か」を判断してから書き始めよう。

複数行を受け取る

- 「1 行目に N、続く N 行に…」という入力は for 文で N 回 ReadLine する

C# Program.cs

```
1  string text = Console.ReadLine();
2  int n = int.Parse(text);
3  int sum = 0;
4  for (int i = 0; i < n; i++)
5  {
6      string line = Console.ReadLine();
7      int number = int.Parse(line);
8      sum += number;
9  }
10 Console.WriteLine(sum);
```

入力例

3
10
20
30

出力

60

POINT

1 行目で「この後何行あるか」を受け取り、その回数だけ繰り返して読む。

条件分岐：if / else if / else

- 条件が **true** のときだけ {} の中を実行する
- 上から順に調べ、最初に当てはまった所だけ実行
- 比較：== != < > <= >=
- かつ：&& または：||

```
C# Program.cs
1  int score = 75;
2  if (score ≥ 80)
3  {
4      Console.WriteLine("A");
5  }
6  else if (score ≥ 60)
7  {
8      Console.WriteLine("B");
9  }
10 else
11 {
12     Console.WriteLine("C");
13 }
```

注意

「等しい」は = ではなく ==。文字列の比較も words[0] == "swap" のように書ける。

復習

繰り返し：for 文と foreach 文

- for：回数を決めて繰り返す
- iで「何回目か」が使える

C# Program.cs

```
1 for (int i = 0; i < 3; i++)  
2 {  
3     Console.WriteLine(i);  
4 }
```

出力

0
1
2

- foreach：配列の中身を順に取り出す
- 番号はいらないときに便利

C# Program.cs

```
1 int[] scores = { 80, 65, 90 };  
2 foreach (int s in scores)  
3 {  
4     Console.WriteLine(s);  
5 }
```

出力

80
65
90

配列

- 同じ型の値をまとめて入れておく箱の列
- 番号（インデックス）は 0 から始まる
- 要素数は 配列名.Length
- 最後の要素は [配列名.Length - 1]

C# Program.cs

```
1 int[] numbers = new int[3];  
2 numbers[0] = 10;  
3 numbers[1] = 20;  
4 numbers[2] = 30;  
5 Console.WriteLine(numbers.Length);  
6 Console.WriteLine(numbers[0] + numbers[2]);
```

出力

```
3  
40
```

注意

要素数 3 の配列に numbers[3] は存在しない（範囲外エラー）。「3 番目」は numbers[2]。

関数（メソッド）

- 処理に名前をつけて、何度でも呼び出せるようにする
- static 戻り値の型 名前(引数) { ... }
- return で結果を返す。返さないなら void
- Paiza では Main と同じ class Program の中に書く

```
C# Program.cs
1  class Program
2  {
3      static int Add(int a, int b)
4      {
5          return a + b;
6      }
7      static void Main()
8      {
9          int result = Add(3, 5);
10         Console.WriteLine(result);
11     }
12 }
```

復習

クラスの軽い復習

- データ（フィールド）と処理（メソッド）をまとめた設計図
- new クラス名() で実体（インスタンス）を作る
- 変数名.フィールド / 変数名.メソッド() で使う
- public をつけると外から使える

出力

ゆうた 80

C# Program.cs

```
1  public class Player
2  {
3      public string Name;
4      public int Hp;
5      public void TakeDamage(int dmg)
6      {
7          Hp -= dmg;
8      }
9  }
10 // Main の中
11 Player p1 = new Player();
12 p1.Name = "ゆうた";
13 p1.Hp = 100;
14 p1.TakeDamage(20);
15 Console.WriteLine(p1.Name + " " + p1.Hp);
```

2

LESSON 2

例題への挑戦

目標：復習した書き方を使って、Paiza の例題を自分の力で解く

演習へ：Paiza の例題を解いてみよう

- LESSON 1 で復習した書き方を使って、実際に Paiza の問題を解いていきます。



フィボナッチ数 C# 編

https://paiza.jp/works/mondai/array_primer/array_primer_fibo/edit?language_uid=c-sharp



【配列の検索】何個ある? Boss C# 編

https://paiza.jp/works/mondai/array_primer/array_primer_search_count_boss/edit?language_uid=c-sharp



【配列に対しての複雑な処理】集団行動 C# 編

https://paiza.jp/works/mondai/array_utilization_primer/array_utilization_primer_group_move/edit?language_uid=c-sharp

POINT

困ったら「入力の受け取り方 一覧」を見返そう。まずは入力例で実行して、出力例と一致するか確かめてから提出。

3

LESSON 3

授業課題

目標：スキルチェックの問題を、自分の力で最後まで解く

スキルチェックに挑戦

- 授業内で取り組み、時間内に終わらなかった問題は宿題とします



授業課題 1

<https://paiza.jp/challenges/560/ready>



授業課題 2

<https://paiza.jp/challenges/766/ready>



授業課題 3

<https://paiza.jp/challenges/890/ready>

POINT

スキルチェックは自分の力で解くこと。困ったら「入力の受け取り方 一覧」を見返そう。